PROJECT REPORT

3D Portfolio Website Project Report

Frontend, Backend, API Integration, and Deployment Support

SUBMITTED BY	Ashish Panday
PROJECT TYPE	Full-Stack Web Development Project
GENERATED ON	March 13, 2026
LIVE WEBSITE	https://ashishpanday.vercel.app/

This report presents the design and development of a full-stack 3D portfolio website that combines immersive frontend presentation with practical backend features such as API routing, validation, contact form handling, storage, and deployment support.

ABSTRACT

This report presents the design and development of a full-stack 3D portfolio website that combines immersive frontend presentation with practical backend features such as API routing, validation, contact form handling, storage, and deployment support.

1. Introduction

This project is a modern full-stack personal portfolio website developed to present academic, technical, and research work in a visually engaging format. The application combines a 3D animated frontend with a functional backend, turning a static portfolio into a complete web application.

The project was designed to demonstrate both creative frontend development and practical backend implementation. It shows not only user interface and animation skills, but also server-side programming, form processing, data validation, API integration, and deployment readiness.

2. Objectives

- To create a visually impressive 3D portfolio website.
- To implement responsive frontend sections for content presentation.
- To add backend functionality for real user interaction.
- To create API routes for portfolio data and message handling.
- To validate contact form input before storing data.
- To prepare the project for deployment on Vercel.
- To make the project suitable as a complete college-level full-stack application.

3. Technologies Used

Frontend Technologies

- React
- Vite
- Tailwind CSS
- GSAP
- Three.js
- React Three Fiber
- Drei
- Lenis
- Motion

Backend Technologies

- Node.js
- Express.js
- CORS
- REST API architecture

Deployment and Storage

- Vercel
- Vercel Functions
- Vercel Blob Storage

4. System Overview

The system is divided into two major parts: the frontend and the backend. The frontend is responsible for the visual presentation of the portfolio, while the backend handles data communication, API responses, validation, and message storage.

The main frontend sections are Hero, About, Services, Works, Contact Summary, and Contact. The backend exposes routes such as /api/portfolio, /api/health, /api/messages, and /api/contact. The frontend communicates with these routes through HTTP requests.

The project also supports two presentation modes: a dynamic 3D React-based experience and a lightweight static HTML version. A visible toggle allows users to switch between the dynamic and static views, making the website flexible for both immersive presentation and minimal reading mode.

5. Frontend Implementation

The frontend was built using React and Vite, with Tailwind CSS for styling and GSAP for animations. Three.js through React Three Fiber and Drei was used to create the 3D visual experience in the Hero section.

A smooth loading experience was implemented using useProgress from Drei, and scroll behavior was enhanced with Lenis. Different sections of the portfolio present the user's research profile, services, projects, and contact information in a responsive layout.

In addition to the animated 3D version, the project includes a static minimal page built with HTML and CSS. The main dynamic website provides a Minimal toggle, and the static page provides a 3D View toggle, so users can move back and forth between the two experiences.

Main Frontend Features

- Animated hero section with 3D graphics.
- Smooth transitions and scroll-based motion.
- Responsive layout across desktop and mobile devices.

- Dynamic services and works sections.
- Dual presentation modes with toggle support between static and dynamic views.
- Interactive contact section with backend integration.
- Frontend fallback data when the backend is unavailable.

6. Backend Implementation

The backend was implemented using Node.js and Express.js for local development. Its purpose is to process requests from the frontend, manage data exchange, validate user input, and support deployment-ready API behavior.

The backend logic was structured into service and storage layers. Shared backend modules were created so the same logic can be used for both local Express routes and deployed Vercel serverless functions.

Main API Endpoints

- GET /api/portfolio: provides profile information, services, and project data.
- GET /api/health: returns backend status, submission count, and storage mode.
- GET /api/messages: returns stored contact form submissions.
- POST /api/contact: receives, validates, and stores contact form messages.

7. Contact Form Functionality

The Contact section includes a real form where users submit their name, email, subject, and message. When the form is submitted, the frontend sends the data to the backend using an API request.

The backend validates each field before saving the message. If any field is invalid, the backend returns field-specific error messages to the frontend. If all fields are valid, the message is stored and a success response is returned to the user.

Validation Rules

- Name must contain meaningful text.
- Email must be in valid email format.
- Subject must not be too short.
- Message must contain sufficient content.

8. Data Storage

During local development, submitted messages are stored in a JSON file. This makes testing simple and allows inspection of saved data during development.

For deployment, the project was upgraded to support Vercel Blob Storage because local file writing is not reliable in serverless hosting environments. This allows the deployed contact form to keep working correctly

after publication.

9. Deployment Support

The project was prepared for deployment on Vercel. Dedicated API handlers were created so that the same backend features can run as Vercel Functions in production.

This means the website can be shared through a live link, and the professor or any other viewer can use the contact form successfully, provided the Vercel project is connected to Blob storage.

10. Project Workflow

- The user opens the website.
- The frontend loads the portfolio sections and requests backend data.
- The backend returns portfolio data and health information.
- The user fills in the contact form and submits a message.
- The backend validates the data and stores the message.
- The frontend displays a success or error response.

11. Key Achievements

- Built a visually rich 3D portfolio website.
- Added a working backend with multiple API endpoints.
- Implemented a real contact form system.
- Added input validation and error handling.
- Integrated frontend and backend successfully.
- Added deployment-ready support for Vercel.
- Supported both local file storage and Vercel Blob storage.

12. Challenges Faced

- The original project was frontend-only, so backend integration had to be added without disturbing the visual structure.
- The contact form initially failed when only the frontend was running and the backend server was not active.
- Local JSON storage was useful for development but unsuitable for permanent cloud deployment.
- The backend needed to be redesigned to work correctly on Vercel using serverless API functions and persistent storage.

13. Conclusion

This project successfully combines modern frontend design with practical backend engineering. The frontend offers a strong visual experience through animation, responsiveness, and 3D graphics, while the backend adds real-world functionality through APIs, validation, storage, and deployment support.

As a result, the project is not only a creative portfolio but also a meaningful full-stack web application suitable for academic presentation and real deployment.

14. Future Improvements

- Admin dashboard to view submitted messages.
- Authentication system for secure access.
- Database integration such as MongoDB or PostgreSQL.
- Automatic email notification after form submission.
- Analytics dashboard for traffic and message insights.
- Content management panel for updating projects dynamically.

15. References

- React Documentation
- Vite Documentation
- Tailwind CSS Documentation
- GSAP Documentation
- Three.js Documentation
- React Three Fiber Documentation
- Node.js Documentation
- Express.js Documentation
- Vercel Documentation
- Vercel Blob Documentation